

FluxiaCRM — API pública (v1) para o Hermes / n8n

Base URL de produção: `https://crm.salestecnologia.com.br` Nos exemplos abaixo, troque `your-crm.example.com` / `<host>` por essa base URL. Autenticação: `Authorization: Bearer <sua_chave>`.

Novidades (30/07): etiquetas (`tags:read` / `tags:write`) e **prioridade** da conversa; **código do cliente** (`customer_codes` , múltiplos por contato) no GET/POST/PATCH de contatos. Ver seções **Tags (labels)** e **Contacts**.

Gerado em 2026-07-30.

Public API (`/api/v1`)

The public API lets you drive your wacrm instance from your own scripts and automations — send messages, manage contacts, launch broadcasts — without going through the dashboard UI.

Status: stable. Authentication, scopes, rate limiting, the messages / contacts / conversations / broadcasts endpoints, and outbound event **webhooks** all ship now.

Authentication

Every request authenticates with an **API key**, sent as a bearer token:

```
Authorization: Bearer wacrm_live_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

Keys are **account-scoped**: a key acts on exactly one account, the one it was created in. There is no cross-account access.

Creating a key

In the dashboard: **Settings** → **API keys** → **New API key**. Only **admins and owners** can create keys.

1. Give the key a name (after the integration that will use it).
2. Grant the **scopes** it needs — nothing more (see below).
3. Copy the key. **The full key is shown exactly once**. wacrm stores only a SHA-256 hash, so it can never be shown again. If you lose it, revoke it and create a new one.

Revoking a key

Settings → **API keys** → **Revoke**. Revocation is effective on the key's next request. Revoked keys stay in the list as an audit trail.

Scopes

A key can do only what its scopes allow — independent of who created it. Grant the minimum.

Scope	Allows
<code>messages:send</code>	Send WhatsApp messages
<code>messages:read</code>	Read messages and delivery status
<code>contacts:read</code>	List and read contacts
<code>contacts:write</code>	Create and update contacts
<code>conversations:read</code>	List and read conversations

<code>conversations:write</code>	Assign / move / close / prioritize conversations
<code>tags:read</code>	List the account tags (labels)
<code>tags:write</code>	Create tags; add/remove them on conversations
<code>deals:read</code>	List and read pipelines and deals
<code>deals:write</code>	Create and move deals (Kanban cards)
<code>tasks:read</code>	List and read tasks
<code>tasks:write</code>	Create and update tasks
<code>agent:read</code>	Read the AI text agent's configuration
<code>agent:write</code>	Configure the AI text agent
<code>members:read</code>	List team members
<code>internal:read</code>	Read internal team channels
<code>internal:write</code>	Create internal channels and post messages
<code>broadcasts:send</code>	Launch broadcast campaigns; list channels
<code>webhooks:manage</code>	Register and manage outbound webhooks

A key with **no scopes** still authenticates and can call `GET /api/v1/me` — useful for verifying a key works.

Response envelope

Every response uses one of two shapes:

```
// success
{ "data": { /* ... */ } }

// failure
{ "error": { "code": "forbidden", "message": "This API key is missing the 'messages:send' scope" } }
```

Branch on `error.code` (stable); `error.message` is for humans and may be reworded.

Status	code	Meaning
401	<code>unauthorized</code>	Missing / malformed / unknown / revoked / expired key
403	<code>forbidden</code>	Valid key, but missing the required scope
429	<code>rate_limited</code>	Per-key rate limit exceeded

400	<code>bad_request</code>	Malformed input
404	<code>not_found</code>	No such resource
500	<code>internal</code>	Server error

Rate limits

Requests are limited **per key: 120 requests per minute**. On a `429`, these headers tell you when to retry:

- `Retry-After` — seconds until the window resets
- `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset`

The limiter is in-memory and **per process**. A single-instance deploy (the common case for a self-hosted fork) is fine as-is. If you scale to multiple instances, swap the limiter for a shared store (Redis/Upstash) — see the note at the top of `src/lib/rate-limit.ts`. The limit is otherwise unenforced across instances.

Endpoints

GET /api/v1/me

Returns the account a key is bound to and the scopes it carries. Requires only a valid key (no scope). Use it to verify a key works and to discover its scopes.

```
curl https://your-crm.example.com/api/v1/me \
-H "Authorization: Bearer wacrm_live_xxx"
```

```
{
  "data": {
    "account": { "id": "...", "name": "Acme Inc" },
    "key": { "id": "...", "scopes": ["messages:send"] }
  }
}
```

POST /api/v1/messages

Send a WhatsApp message. Scope: `messages:send`. Target it in one of two ways:

- `to` — an **E.164 number**; the endpoint finds-or-creates the contact + 1:1 conversation, then sends.
- `conversation_id` — send straight to an existing conversation. This is the **only way to reply in a group** (groups have no phone number). Read the group's `conversation_id` from `GET /api/v1/conversations`.

```
curl -X POST https://your-crm.example.com/api/v1/messages \
  -H "Authorization: Bearer wacrm_live_xxx" \
  -H "Content-Type: application/json" \
  -d '{ "to": "+14155550123", "type": "text", "text": "Hi 🙌" }'
```

`type` is `text` (default), `template`, or a media kind (`image` / `video` / `document` / `audio`). Media needs `media_url` (and optional `filename`); `text` doubles as the caption. `template` needs a `template` object:

```
{
  "to": "+14155550123",
  "type": "template",
  "template": {
    "name": "order_update",
    "language": "en_US",
    "params": ["A123"]           // positional body vars, or a structured object
  },
  "reply_to_message_id": "<uuid>" // optional; must be in the same conversation
}
```

Response (201):

```
{
  "data": {
    "message_id": "...",
    "whatsapp_message_id": "wamid...",
    "conversation_id": "...",
    "contact_id": "...",
    "contact_created": true
  }
}
```

Domain error codes beyond the table above: `whatsapp_not_configured` (400), `meta_error` (502 — the request reached Meta and it rejected the send), `template_malformed` (500).

GET /api/v1/contacts

List contacts, newest first. Scope: `contacts:read` . Paginated (see [Pagination](#)). Optional filters: `?search=` (matches name or phone) and `?tag=<tagId>` .

```
{
  "data": [
    {
      "id": "...", "phone": "+14155550123", "name": "Jane Doe",
      "email": null, "company": "Acme", "avatar_url": null,
      "customer_codes": ["31768", "31770"],
      "tags": [{ "id": "...", "name": "vip", "color": "#3b82f6" }],
      "created_at": "...", "updated_at": "..."
    }
  ],
}
```

```

"meta": { "next_cursor": "..." }
}

```

POST /api/v1/contacts

Create a contact. Scope: `contacts:write` . `phone` (E.164) is required; `name` , `email` , `company` , `tags` (an array of tag names, created if missing) and `customer_codes` (an array of ERP codes — múltiplos por contato) are optional. **Find-or-create by phone:** an existing match returns `200` with the existing contact; a new contact returns `201` . The response body is the serialized contact (same shape as the list rows above).

GET / PATCH /api/v1/contacts/{id}

Read or update one contact. Scopes: `contacts:read` / `contacts:write` . `PATCH` updates only the fields you send (`name` , `email` , `company`); pass `tags` (an array of tag names) to replace the contact's tags, and `customer_codes` (an array of strings) to replace the ERP codes. A contact in another account returns `404` .

GET /api/v1/conversations

List conversations, newest first. Scope: `conversations:read` . Paginated. Optional filters: `?status=` (`open` / `pending` / `closed`) and `?contact_id=` . Each conversation embeds its contact + tags.

GET /api/v1/conversations/{id}

Read one conversation. Scope: `conversations:read` . `404` if it belongs to another account.

GET /api/v1/conversations/{id}/messages

List a conversation's messages, newest first. Scope: `messages:read` . Paginated. Each message includes its `direction` (`inbound` / `outbound`), `status` (delivery state), `whatsapp_message_id` , and `content_*` . The conversation is verified to belong to your account first (`404` otherwise).

POST /api/v1/broadcasts

Launch a template broadcast to a list of recipients. Scope: `broadcasts:send` . The broadcast + its recipient rows are persisted immediately and the sends fan out in the background, so the call returns fast — poll `GET /api/v1/broadcasts/{id}` for progress.

```

curl -X POST https://your-crm.example.com/api/v1/broadcasts \
  -H "Authorization: Bearer wacrm_live_xxx" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "July promo",
    "template_name": "promo_july",
    "template_language": "en_US",
    "recipients": [
      { "to": "+14155550123", "params": ["Jane"] },
      { "to": "+14155550124" }
    ]
  }'

```

Recipients are capped at **1000 per request** — split larger sends. Invalid phone numbers are dropped and counted as **rejected**. Response (202):

```
{
  "data": {
    "broadcast_id": "...",
    "status": "sending",
    "total_recipients": 2,
    "accepted": 2,
    "rejected": 0
  }
}
```

GET /api/v1/broadcasts/{id}

Broadcast status + counts. Scope: **broadcasts:send**. **status** moves **sending** → **sent**; **delivered_count** / **read_count** keep climbing as Meta delivery webhooks arrive. **404** for another account's broadcast.

PATCH /api/v1/conversations/{id}

Assign, move, close, or **prioritize** a conversation. Scope: **conversations:write**. Accepts any of: **assigned_agent_id** (member id, or **null** to unassign), **sector_id** (sector id, or **null** for the general queue), **status** (**open** / **pending** / **closed**), **priority** (**none** / **low** / **medium** / **high** / **urgent**).

```
# Flag a thread as urgent for the team
curl -X PATCH https://<host>/api/v1/conversations/{id} \
  -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \
  -d '{"priority": "urgent" }'
```

GET /api/v1/conversations and **GET /api/v1/conversations/{id}** include the current **priority** and the contact's **tags** on each row.

Tags (labels)

Account-level labels that live on the contact and show on the conversation card. Scopes: **tags:read** / **tags:write**.

- **GET /api/v1/tags** — list the account's tags ({ **id**, **name**, **color** }).
- **POST /api/v1/tags** — create a tag. Body: { "name": "pago", "color": "#10b981" } (**color** optional).
Idempotent by name (case-insensitive): re-running an n8n "setup" workflow returns the existing tag (**200**) instead of duplicating it; a brand-new tag returns **201**.
- **GET /api/v1/conversations/{id}/tags** — tags on this thread's contact.
- **POST /api/v1/conversations/{id}/tags** — add a tag to the thread. Body is either { "tag_id": "..." } or { "name": "pago", "color": "#10b981" } — a name that doesn't exist yet is created, so Hermes can label freely.
Returns the contact's full tag set.
- **DELETE /api/v1/conversations/{id}/tags/{tagId}** — remove a tag (idempotent).

```
# Hermes labels a thread (creating the tag if needed)
curl -X POST https://<host>/api/v1/conversations/{id}/tags \
  -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \
  -d '{"name": "pago" }'
```

Deals & pipelines

Scopes: `deals:read` / `deals:write` .

- `GET /api/v1/pipelines` — list pipelines and their ordered stages.
- `GET /api/v1/deals` — list cards. Filters: `?pipeline_id=` , `?stage_id=` , `?contact_id=` , `?status=` .
- `POST /api/v1/deals` — create a card. Only `title` is required (pipeline/stage default to the account's first). Optional: `value` , `currency` , `contact_id` , `notes` , `expected_close_date` .
- `GET` / `PATCH /api/v1/deals/{id}` — read or update. **Moving a card** = `PATCH` with `stage_id` (and `pipeline_id` if changing board).

Tasks

Scopes: `tasks:read` / `tasks:write` .

- `GET /api/v1/tasks` — list tasks, newest first (paginated). Filters: `?status=` (`open` / `done` / `cancelled`), `?contact_id=` , `?deal_id=` , `?assigned_to=` .
- `POST /api/v1/tasks` — create. Only `title` is required. Optional: `description` , `due_at` (ISO 8601), `type` , `contact_id` , `deal_id` , `assigned_to` , `status` .
- `GET` / `PATCH /api/v1/tasks/{id}` — read or update. Mark done with `{ "status": "done" }` ; `PATCH` touches only the fields you send.

```
curl -X POST https://your-crm.example.com/api/v1/tasks \
  -H "Authorization: Bearer wacrm_live_xxx" -H "Content-Type: application/json" \
  -d '{"title": "Follow up with Jane", "due_at": "2026-08-01T14:00:00Z", "contact_id": "..." }'
```

AI text agent

Scopes: `agent:read` / `agent:write` . The account's single AI configuration (provider, model, system prompt, own API key) that powers AI-drafted replies, the auto-reply bot and the Playground.

- `GET /api/v1/agent` — read the config (`configured:false` when none). The API key is never returned.
- `PUT /api/v1/agent` — configure. Body: `provider` (`openai` / `anthropic`), `model` , `api_key` (omit to keep the stored one), and optional `system_prompt` , `is_active` , `auto_reply_enabled` , `auto_reply_max_per_conversation` (1–20). The key is **validated with the provider** before it's stored (AES-256-GCM encrypted).

```
curl -X PUT https://your-crm.example.com/api/v1/agent \
  -H "Authorization: Bearer wacrm_live_xxx" -H "Content-Type: application/json" \
  -d '{"provider": "openai", "model": "gpt-4o-mini", "api_key": "sk-...",
    "system_prompt": "Você é o atendente da Acme...", "is_active": true }'
```


Channels, members & internal chat

- `GET /api/v1/channels` · `GET /api/v1/channels/{id}` — list WhatsApp channels (id, name, provider, status). Scope: `broadcasts:send`.
- `GET /api/v1/members` — list team members (id, name). Scope: `members:read`.
- `GET / POST /api/v1/internal/channels` — list / create internal team channels (`{ name, is_private?, member_ids? }`). Scopes: `internal:read` / `internal:write`.
- `GET / POST /api/v1/internal/channels/{id}/messages` — read / post internal messages.

More broadcasts

- `POST /api/v1/broadcasts/text` — text/media campaign on a non-official (WAHA) channel. Body: `channel_id`, `name`, `body_text`, optional `media_url` / `media_type` / `media_filename`, and `recipients` (list of `to`) or `contact_ids`.
- `GET /api/v1/broadcasts/{id}/recipients` — per-recipient status.
- `POST /api/v1/broadcasts/{id}/pause` · `/resume` · `/cancel` — control a running campaign.

Pagination

Every list endpoint pages the same way. Request a page size with `?limit=` (default 50, max 100) and read the next page with the opaque `meta.next_cursor` from the previous response:

```
GET /api/v1/contacts?limit=50
→ { "data": [ ... ], "meta": { "next_cursor": "eyJ..." } }

GET /api/v1/contacts?limit=50&cursor=eyJ...
→ { "data": [ ... ], "meta": { "next_cursor": null } } // last page
```

Cursors are keyset-based (stable under concurrent inserts). Pass the cursor back verbatim — don't parse it. `next_cursor: null` means the last page.

Webhooks

Rather than polling, register an endpoint and wacrm will POST to it when things happen in your account. **Migration required:** apply `supabase/migrations/028_webhook_endpoints.sql`.

Events

Event	Fires when
<code>message.received</code>	An inbound message arrives from a contact
<code>message.status_updated</code>	A message you sent changed delivery status
<code>conversation.created</code>	A new conversation is opened for a contact

Managing endpoints

All under scope `webhooks:manage` .

- `POST /api/v1/webhooks` — register `{ "url": "https://...", "events": ["message.received"] }` . `url` must be `https://` . The response includes `secret` exactly once — store it to verify signatures; wacrm keeps only an encrypted copy.
- `GET /api/v1/webhooks` — list your endpoints (never returns the secret).
- `GET /api/v1/webhooks/{id}` — read one.
- `PATCH /api/v1/webhooks/{id}` — update `url` , `events` , or `is_active` (re-enabling clears the failure counter).
- `DELETE /api/v1/webhooks/{id}` — remove one.

```
curl -X POST https://your-crm.example.com/api/v1/webhooks \
  -H "Authorization: Bearer wacrm_live_xxx" \
  -H "Content-Type: application/json" \
  -d '{ "url": "https://example.com/hooks/wacrm", "events": ["message.received"] }'
# → 201 { "data": { "id": "...", "url": "...", "events": [...], "secret": "whsec_..." } }
```

Delivery payload

Every delivery is a POST with this envelope; `id` is a unique per- delivery uuid you can dedupe on, and `data` varies by `event` :

```
{
  "id": "8f3c...",
  "event": "message.received",
  "occurred_at": "2026-07-01T12:00:00.000Z",
  "account_id": "...",
  "data": { /* per-event, see below */ }
}
```

`data` by event:

```
// message.received
{ "conversation_id": "...", "contact_id": "...", "whatsapp_message_id": "wamid...", "content_type": "text" }
// conversation.created
{ "conversation_id": "...", "contact_id": "..." }
// message.status_updated
{ "whatsapp_message_id": "wamid...", "conversation_id": "...", "status": "delivered" }
```

Headers: `X-Wacrm-Event` , `X-Wacrm-Webhook-Id` , and `X-Wacrm-Signature` .

Verifying the signature

`X-Wacrm-Signature: t=<unix_seconds>,v1=<hex>` where `v1 = HMAC-SHA256(secret, "${t}.${rawBody}")` .

Recompute it over the **raw request body** and compare in constant time; reject if `t` is more than a few minutes old (replay protection).

```
const [, t, v1] = header.match(/t=(\d+),v1=([0-9a-f]+)/);
const expected = crypto.createHmac('sha256', secret)
  .update(`${t}.${rawBody}`).digest('hex');
const ok = crypto.timingSafeEqual(Buffer.from(expected), Buffer.from(v1));
```

Delivery semantics

Delivery is **best-effort**: a single attempt per event with a short timeout, and **redirects are not followed**.

`message.status_updated` covers messages wacrm stores (inbox + API sends), not broadcast-only sends, and — because providers re-send and re-order status callbacks — the same status may arrive more than once or out of order; **dedupe on `id` and don't assume ordering**. Each consecutive failure increments `failure_count`; after enough consecutive failures the endpoint is auto-disabled (`is_active: false`) — re-enable it with `PATCH` (which resets the counter). Durable retry-with-backoff (a delivery queue) is a future enhancement; today, treat missed deliveries as possible and reconcile with the read endpoints when it matters.

Target restrictions (SSRF). The `url` must be `https://` and must resolve to a public address — requests to `localhost`, private/RFC1918 ranges, link-local (incl. cloud metadata `169.254.169.254`), and similar internal targets are refused at delivery time.

Roadmap

The public API covers messaging, contacts, conversations, deals & pipelines, tasks, the AI text agent, channels, members, internal chat, broadcasts (template + text/WAHA), and outbound webhooks. Not yet exposed via the API (done in the dashboard for now): creating **flows** and **automations**, and a delivery queue for webhook retries.